

Template Engine mit node.js

- **Ziel:** Saubere Trennung von Darstellung und Inhalt
→ Trennung von HTML und Node.js-Code
- **Lösung:** Template-Engine
- **Idee:**
 - > Einlesen einer HTML-Vorlage (Template) mit vorgesehenen Platzhaltern
 - > Ersetzung der Platzhalter durch Attributwerte eines passenden Objekts
- Muss ich die Template-Engine selber implementieren?

NEIN!

Die elementare Nutzung (Quelle in der Abbildung)

```
const person = {  
  firstName: "Volker",  
  lastName: "Sander",  
  blogURL: "http://sandersblog.org"  
};  
let template = "<h1>{{firstName}}  
{{lastName}}</h1>Blog: {{blogURL}}";  
let html = Mustache.to_html(template,  
person);
```

Automatisches Escaping:

Mit diesen doppelten geschweiften Klammern wird automatisch HTML escaped

Aus dem Template durch Ersetzung erstelltes HTML

Inhalt



Template Engine

Mustache.js

```
<div>
  
  <p>{{name}}</p>
</div>
```

Darstellung



```
import mustache from 'mustache'

//template laden ...

inhalt={
  'name': 'Klaus',
  'avatar': 'BudSpencer.jpg'
}

let html =
mustache.render(template,
inhalt)
```

Inhalt



Template laden?

```
import mustache from 'mustache'

//template laden ...

inhalt={
  'name': 'Klaus',
  'avatar': 'BudSpencer.jpg'
}
mustache.render(template, inhalt)
```

Template Engine

Mustache.js

Template laden?

Express hilft!

Bei node.js mit
ESmodulen (import) ist
__dirname und
__filename nicht mehr
gesetzt, wir müssen dies
programmatisch erzeugen

```
import express from 'express'
import { fileURLToPath } from 'url';
import path from 'path';

const app = express();

const router = express.Router()

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
import mustacheExpress from 'mustache-express'
app.set('view engine', 'mustache')
app.engine('html', mustacheExpress());
app.set('views', path.join(__dirname , 'views'))

router.get('/', function(req, res){
  inhalt={
    'name': 'Klaus',
    'avatar': 'BudSpencer.jpg'
  }
  res.render('tpl_datei_views.html', inhalt)
})
```

Gibt das generierte
html direkt zurück

Es gibt verschiedene Template Engines, nicht alle passen

- Nicht alle implementieren die für Express notwendige Funktion `__express(filePath, options, callback)`
- Wir hatten deshalb auch `mustache-express` verwendet, welches für `express` gemacht wurde
- Das Package `consolidate` bietet eine Zwischenschicht für verschiedene Template-Engines an. Es sorgt dafür, dass alle gängigen Template-Engines eine gemeinsame Schnittstelle bereitstellen, sodass sie nahtlos mit Express verwendet werden können

- Beispiel:

```
import consolidate = from 'consolidate'
import express from 'express'
const app = express();
app.engine('html', consolidate.mustache)
app.set('view engine', 'html')
```

Zustand des Node.js-Skriptes geht nach Ausführung verloren

- Variablen sind nur für einen Aufruf gültig

Wie können mehrstufige Operationen/Transaktionen vorgenommen werden?

- Der Server muss erkennen, dass der nächste Aufruf der Seite in einem Kontext geschieht
- ISO/OSI, Schicht 5? Session?
 - > Nein, im HTTP-Protokoll ist das die Aufgabe der Applikation!

Persistente JS Skripte

- Parameter, der den aktuellen „Schritt“ speichert
- Cookies & Sessions bieten die Möglichkeit den Inhalt von Variablen länger zu speichern



Cookies bieten die Möglichkeit clientseitig Daten zu speichern

- Benutzer muss sich nicht mehrfach anmelden
- Aber auch Tracking des Nutzerverhaltens möglich
- Cookies werden im Header der Seite zurückgeliefert
- Das Setzen von Cookies muss daher erfolgen bevor Inhalt ausgegeben wird

- Beispiel:

```
HTTP/1.1 200 OK
```

```
Content-type: text/html
```

```
Set-Cookie: name=value
```

```
Set-Cookie: foo=bar; Expires=Wed, 09 Jun 2021 10:18:14 GMT
```

```
(Inhalt der Seite)
```

Speicherung des Cookies als „Datei“ auf dem Client

- Tatsächlich meist mit Hilfe von Datenbanken realisiert

Client sendet beim nächsten Aufruf der Webseite das Cookie mit

- Nur die Webseite, die das Cookie gesetzt hat, kann das Cookie lesen

- Beispiel:

```
GET / HTTP/1.1
Host: www.example.org
Cookie: name=value; foo=bar
...
```

Setzen eines Cookies in express

- `res.cookie('name', 'value' [, options])`
 - speichert ein Cookie mit dem Namen `name` und den Wert `value`
 - `value` kann ein String oder ein Objekt (Serialisierung zu JSON) sein
 - `options` kann verschiedene Properties haben:
 - einige Beispiel-Properties:
 - `expires`: Verfallsdatum des Cookies
 - `path`: gibt an von welchen Verzeichnissen das Cookie gelesen werden kann
 - `domain`: gibt an an welche Domains das Cookie ausgeliefert werden soll
 - `httpOnly`: kennzeichnet das Cookie als nur für den Webserver zugänglich
 - ...
- Beispiel:

```
res.cookie('rememberme', '1',  
  { expires: new Date(Date.now() + 900000),  
    path: '/' })
```

- > Cookie „rememberme“ mit dem Wert „1“ läuft in 15 Minuten ab und kann von allen Verzeichnissen in der Domain aus gelesen werden.

Weitere Informationen: <http://expressjs.com/de/4x/api.html#res.cookie>

Lesen eines Cookies, das der Client gesendet hat

- Middleware: [cookie-parser](#) notwendig

```
import express from 'express'
import cookieParser from 'cookie-parser'

const app = express()
app.use(cookieParser())
```

- Liefert ein cookies-Objekt mit den Keys des Cookies
- Beispiel:

```
req.cookies.rememberme
```

- kann **nicht im selben Skriptdurchlauf** gespeichert und wieder gelesen werden
 - > Cookie wird erst beim Senden des Clients im Req-Objekt eingetragen
 - > Erst der nächste Skriptdurchlauf kann das Cookie lesen

Weitere Informationen: <http://expressjs.com/de/4x/api.html#req.cookies>

Cookies

- speichern alle Informationen clientseitig ab
- Daten können vom Benutzer manipuliert werden

Sessions

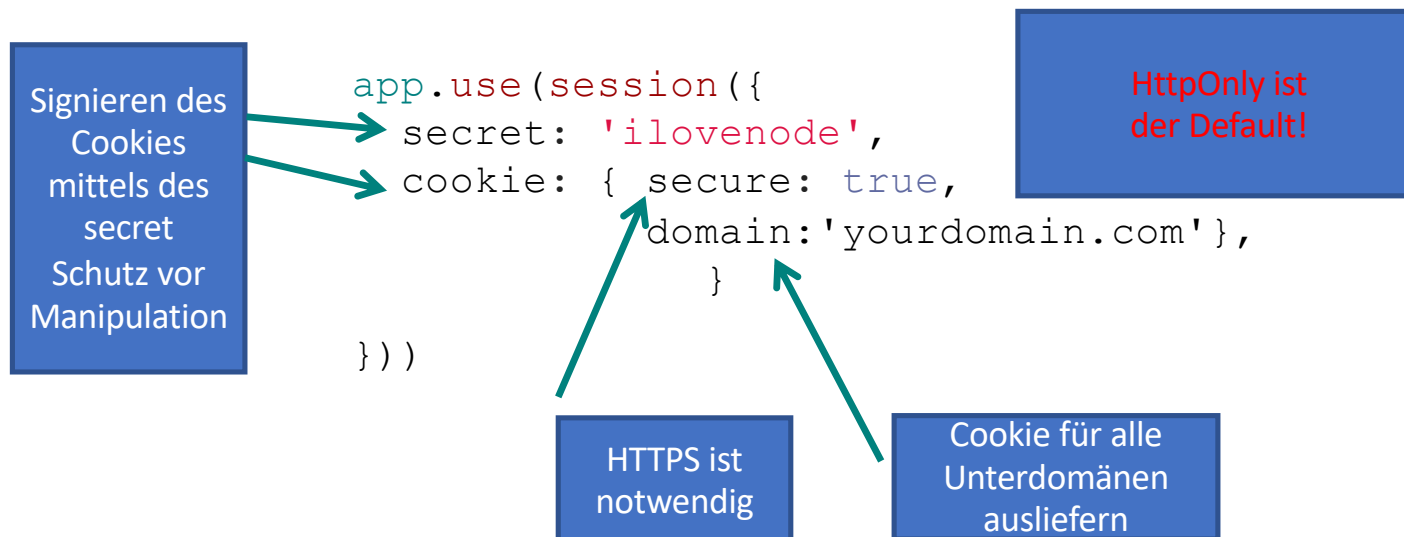
- speichern die Sitzungsdaten serverseitig ab
- Client erhält nur eine Session-ID (als Cookie)
 - > Um das Setzen des Cookies kümmert sich Node
- Daten, die per Session gespeichert werden, werden auch nur auf dem Server gespeichert
 - > keine Datenmanipulation möglich
 - > komplexe Variablen (Arrays, Objekte) sind möglich

Benutzung in Node.js

- Middleware [express-session](#) notwendig

```
import session from 'express-session'
```

- Aktivierung des Session-Features per `app.use`



Weitere Informationen: <https://www.npmjs.com/package/express-session>

- **Beenden einer Session**

```
req.session.destroy()
```

- löscht `req.session`
- Mit Änderung des `secret` wird auch eine neue Session gestartet

Zustände

Sessions (mit require Notation)

```
app = express();

// using express-session
app.use(require('express-session')({

  name: '_es_demo', // The name of the cookie
  secret: '1234', // The secret is required, and is used for signing cookies
  resave: false, // Force save of session for each request.
  saveUninitialized: false // Save a session that is new, but has not been modified

}));

// single path for root
app.get('/', function (req, res) {

  // simple count for the session
  if (!req.session.count) {
    req.session.count = 0;
  }
  req.session.count += 1;

  // respond with the session object
  res.json(req.session);

});
```

Default ist True. Dies erzeugt ggf. unnötigen Aufwand

Wird für Privacy Compliance genutzt. Verhindert, dass Cookies im Browser gesichert werden bevor der Benutzer sein OK gegeben hat. Default ist (leider): True und ist mittlerweile Deprecatated

Zustände mit import

```
// -----  
// 1 Import von express-session  
// -----  
import session from 'express-session';  
  
// -----  
// 2 Session-Middleware konfigurieren & einbinden  
// -----  
app.use(  
  session({  
    name:      '_es_demo',  
    secret:    '1234',      // <-- immer ein langer, zufälliger String  
    resave:    false,      // verhindert unnötiges Resave, wenn nichts geändert wurde  
    saveUninitialized: false, // verhindert das Anlegen leerer Sitzungen  
  })  
)
```

Dynamische Webseiten

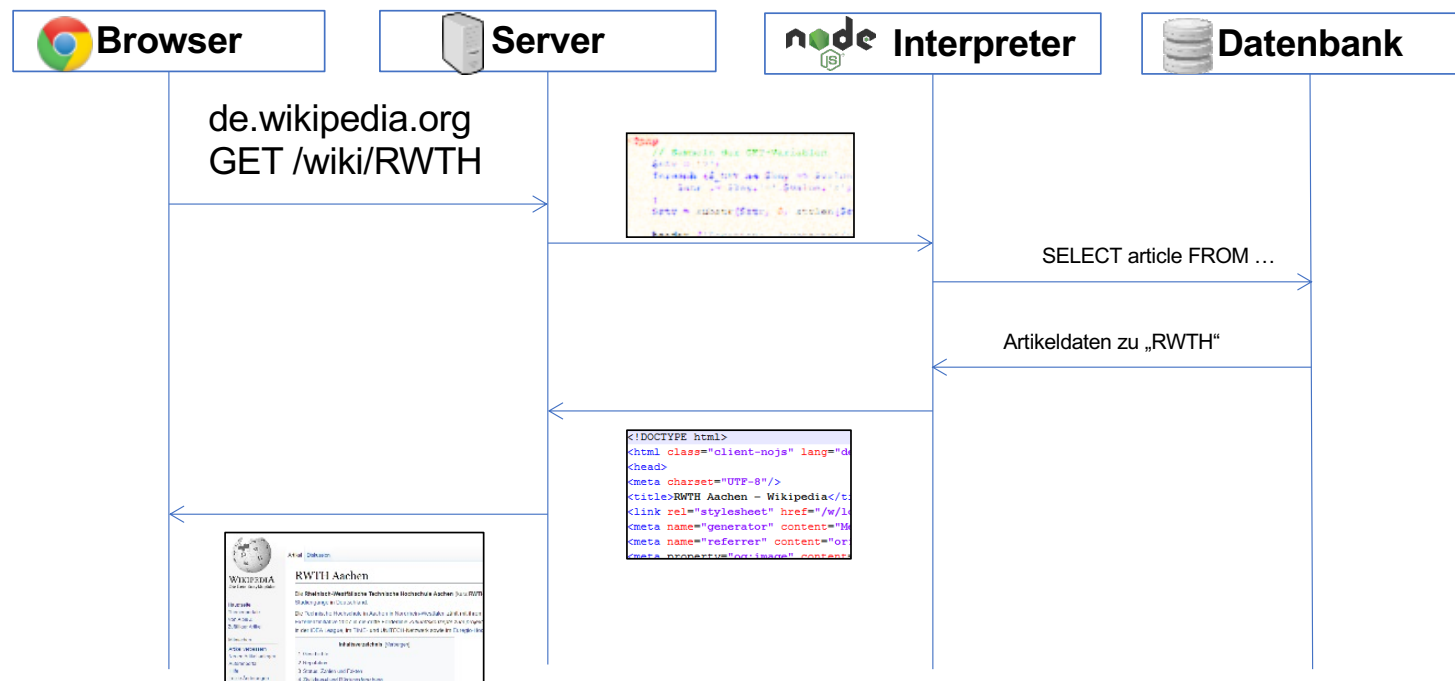
<https://nodejs.org/>

Aufruf einer statischen Webseite (vereinfacht)



Dynamische Webseiten mit node.js

Aufruf einer dynamischen Webseite (vereinfacht)



Beispiel: Login-Formular

```
<form action="do?q=login" method="post">
  <input type="text" name="username" />
  <input type="password" name="pw" />
  <input type="submit" value="Login" />
</form>
```

- Übermittelt Parameter „username“ und „pw“ per POST-Request (hierbei werden die Daten **urlencoded** übergeben, z.B. durch Verwenden der **%xx-Notation** für Sonderzeichen, z.B. **%20** für +

GET-Request

- Übertragung der Daten über die Adresszeile
- Parameter werden durch das Zeichen ? In der URL eingeleitet
- Parameter werden in express im Objekt `req.query` gespeichert
- Nicht geeignet zur Übertragung großer Datenmengen oder sensibler Daten

- Beispiel: `index.html?id=5&foo=bar`

> Zugriff im node.js-Skript:

```
req.query.id // 5
```

```
req.query.foo // bar
```

POST-Request

- Übertragung der Daten im HTTP-Body
 - > Daten sind nicht in der URL sichtbar

- Zugriff auf POST-Daten in express:

```
req.body.text // liefert POST-Parameter text
```

Validierung der Nutzereingaben

- Serverseitige Prüfung von Nutzerdaten ist zwingend erforderlich
- „Never trust the user“



Check- und Filter-Funktionen

- Third Party Node.js Modul: [express-validator](#)
- Validierung der Eingaben

```
check([field, message])
```

- > `field`: zu validierende (Array von) Strings
- > `message`: eigens spezifizierte Fehlermeldung
- > Rückgabe: Validation Chain
 - Hierbei handelt es sich um eine Middleware, die an die Route-Handler übergeben werden sollte
 - Anschaulich werden so viele Checks nacheinander ausgeführt

Dynamische Webseiten

Validierung von Eingaben

Ausgangssituation:

- Einfache Route, die Formdaten verarbeiten soll
 - `name`: Name einer Person (String)
 - `email`: Gültige Mail-Adresse nach RFC5322
 - `age`: Alter der Person als Zahl

Beispiel ohne Überprüfung:

```
import express from 'express';
const app = express();

// convert special characters
app.use(express.urlencoded({extended: true}));

// set POST route
app.post('/form', (req, res) => {
  console.log(`Name: ${req.body.name}, E-Mail: ${req.body.email}, Alter: ${req.body.age}`);
  res.json(req.body);
});
```

Middleware zu Verarbeitung von POST (und PUT) Formulardaten, kodiert mit %xx für Sonderzeichen. Extended verwendet eine Bibliothek für komplexere Objekte (Arrays,...)
Use ohne Pfadangabe: für alle URLs



Dynamische Webseiten

Validierung von Eingaben

```
import express from 'express';
import { check, validationResult, matchedData } from 'express-validator';
const app = express();

// convert special characters
app.use(express.urlencoded({ extended: true }));

// middleware: error handling
function validationError(req, res, next) {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(422).json({ errors: errors.array() }); // 422 Unprocessable Entity
  } else {
    next();
  }
}

// set POST route with validation
app.post('/form', [
  check('name').notEmpty().withMessage('Name is required').isLength({ max: 50 }),
  check('email').isEmail(),
  check('age').optional().isInt({ min: 0, max: 100 }),
  validationError, (req, res) => {
    const { name, email, age } = matchedData(req, { includeOptionals: true });
    console.log(`Name: ${name}, E-Mail: ${email}, Alter: ${age}`);
    res.json({ name, email, age });
  })
], (req, res) => {
  // ...
});
```

Array von Middleware-Funktionen, die ausgeführt werden, deshalb auch `next` in `validationError`. Check liefert eine "validation-Chain" als Middleware, die damit filtert (und bei sanitize-Funktionen korrigiert)

Dynamische Webseiten

Validierung von Eingaben

Beispiel mit Validierung & Bereinigung:

```
// set POST route with validation & sanitizing
app.post('/form', [
  check('name').trim().notEmpty().withMessage('Name is required').isLength({ max: 50 }),
  check('email').isEmail().normalizeEmail(),
  check('age').optional().isNumeric({ min: 0, max: 100 }).toInt(),
  validationResult, (req, res) => {
    console.log(`Name: ${req.body.name}, E-Mail: ${req.body.email}, Alter: ${req.body.age}`);
    res.json(req.body);
  });
```

- Bereinigung der Eingaben

```
sanitize*(fields).[one ore more sanitizer]
```

- > `fields`: zu validierende (Array von) Strings
- > *: **Body**, **Cookie**, **Param**, **Query**
- > Middleware, die * manipuliert

```
buildSanitizeFunction(locations)
```

- > `locations`: Request Locations (z.B. ['body', 'query'])

Chaining wird verwendet, um weitere Sanitizer anzuwenden:

Verfügbare Sanitizer: <https://github.com/validatorjs/validator.js#sanitizers>

Beispiel

```
// escapes HTML entities and cuts whitespaces
app.post('/register',
  sanitizeBody('user').escape().trim(),
  (req, res, next) => {
    console.log(req.body.user);
    console.log(req.body.pw); // not sanitized
  }
);
```

Hinweis: Bei obigem escape() handelt es sich um das escape des Sanitizers und nicht etwa um das veraltete escape() als globales Objekt

(https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Global_Objects/escape)

Sichere Speicherung von Passwörtern

- Passwörter niemals im Klartext speichern
 - > Gefahr, dass die Datenbank in falsche Hände gerät
 - > Nutzer verwenden oft das gleiche Passwort auf mehreren Seiten
- Besser: Hash-Werte speichern
- Third Party Node.js Modul: [argon2](#)
 - > `npm install argon2`

Hinweis:
asynchrone Funktion –
wir lernen `await` dafür
später noch kennen

- Beispiel: `console.log(await argon2.hash('superPW123'))`

```
// liefert z.B.  
$argon2id$v=19$m=65536,t=3,p=4$UaZWj+XshKBArAhpVDv28A$eSAXDk8dWRp+Wyqa7VtnqsyF1g1oB8vowmfXN31bJtA  
  
// oder  
$argon2id$v=19$m=65536,t=3,p=4$afoGLstluVzYhFNsuUF1Dg$905sUdeFKaVv/AQoJd6Hg0n+pNk6uHyBAoUGE1Jxwug  
  
// oder  
$argon2id$v=19$m=65536,t=3,p=4$+Kj+Q7yAhFticKRikN5J0Q$tFQE7bAWU8+fjMQKEHktwjRGliWl6QEOsfBWBv8FEhc
```

Algorithmus

Version

Speicher, Iterationen,
Threads

Salt (Base64-codiert)

Hash (Base64-codiert)

- Gleicher Funktionsaufruf liefert unterschiedliche Werte?
 - > **Argon2 erzeugt automatisch einen zufälligen Salt**

Passwort-API

- Einfache Möglichkeit Passwörter sicher zu speichern

```
argon2.hash(data, [, options])
```

> data: das Passwort

> options: Konfigurationsobjekt, z.B.

```
{ memoryCost: 4096, // Speicherverbrauch in KiB
  parallelism: 3,    // Anzahl Threads
  timeCost: 4        // Anzahl Runden
}
```

> Beispiel: `await argon2.hash('superPassword', {memoryCost: 4096, parallelism: 3, timeCost: 4});`

- Überprüfung ob Passwort korrekt ist:

```
argon2.verify(hash, data)
```

> hash: gespeicherter Argon2-Hash

> data: zu überprüfendes Passwort

> Beispiel: `await argon2.verify(hash, 'superPassword')`

Achtung

- Bei `bcrypt.hash` und `bcrypt.compare` handelt es sich um **asynchrone** Funktionen
- D.h. diesen sollte in Kombination mit `await` in einer mit `async` notierten Funktion aufgerufen werden (siehe später)

Alternative

- Synchroner Variante:
 - > `bcrypt.hashSync`
 - > `bcrypt.compareSync`

Zu bevorzugen ist die asynchrone Variante, damit die Event Loop nicht blockiert wird!

Siehe dazu auch: <https://jinoantony.com/blog/async-vs-sync-nodejs-a-simple-benchmark>

Schädliche Module

- jeder kann Module per npm veröffentlichen und konsumieren
- dabei können schädliche Module in Umlauf geraten
- Gefahr besteht, dass man diese versehentlich per npm installiert
- Beispiel **Typosquatting**: socketio anstelle von socket.io
 - socket.io ist das verbreitetste Modul für Echtzeit-Webanwendungen
 - socketio (ähnlich benannt) → könnte Schadcode enthalten
- auch bekannte Module können verwundbar sein (z.B. durch ein Update)
 - Empfehlung: Version des Moduls in der package.json fixieren
 - [npm-audit](#) nutzen: Projekt nach Verwundbarkeiten durchsuchen

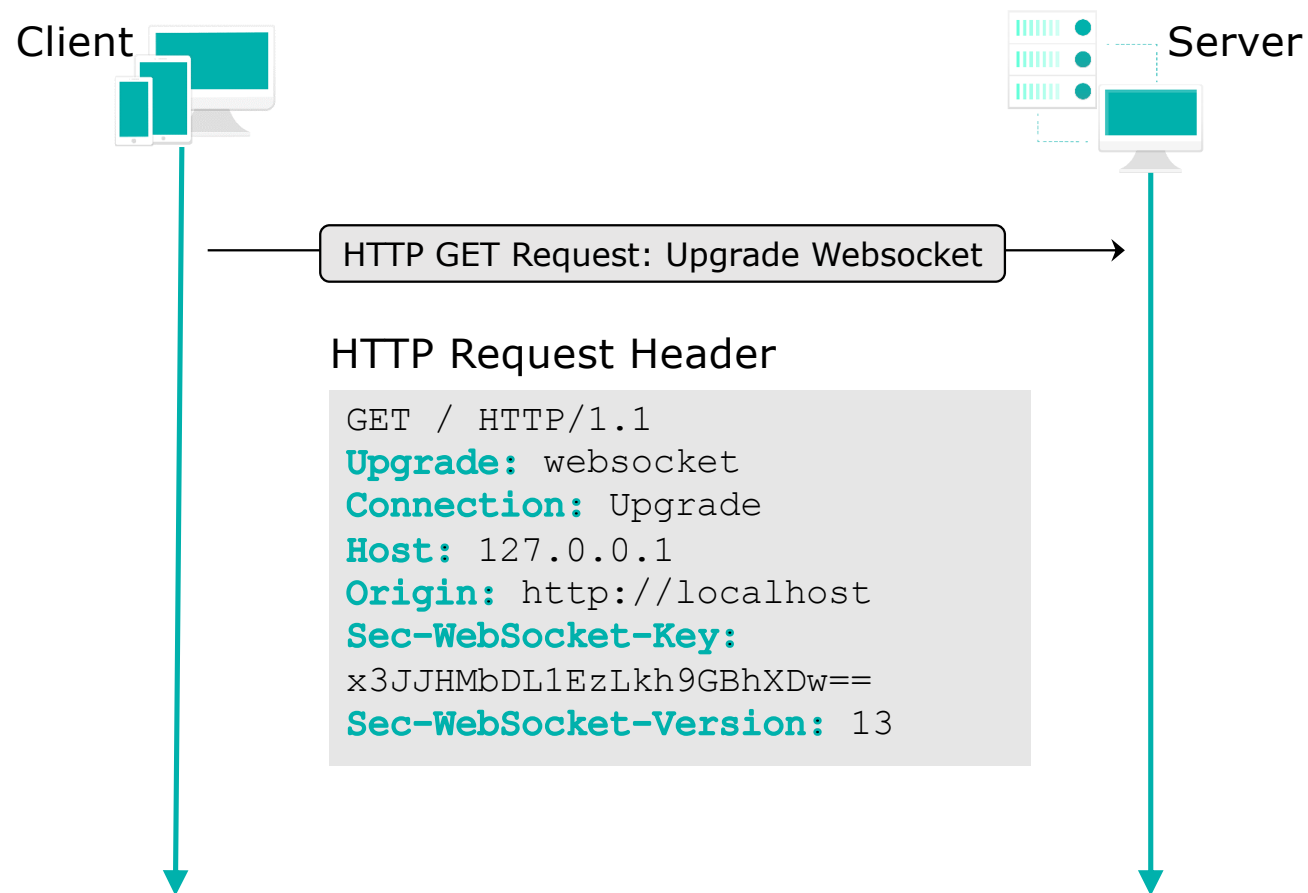
BE
CAREFUL!

Was sind Websockets?

- TCP basiertes Protokoll, welches eine bidirektionale Verbindung zwischen Webbrowser und Server ermöglicht
- nur **ein** Verbindungsaufbau notwendig, dieser geschieht über HTTP und nutzt ähnliche Mechanismen die beim Upgrade auf eine HTTP/2 Verbindung genutzt werden
- TCP-Verbindung wird offengehalten
 - Im Gegensatz zu reinem HTTP: Serverantwort setzt Anfrage durch den Client voraus (Anfrage-Antwort-Prinzip)
- Anwendung: Echtzeitanwendungen
 - Chats
 - Spiele
 - Server-Monitoring
- wird von allen gängigen Browsern unterstützt

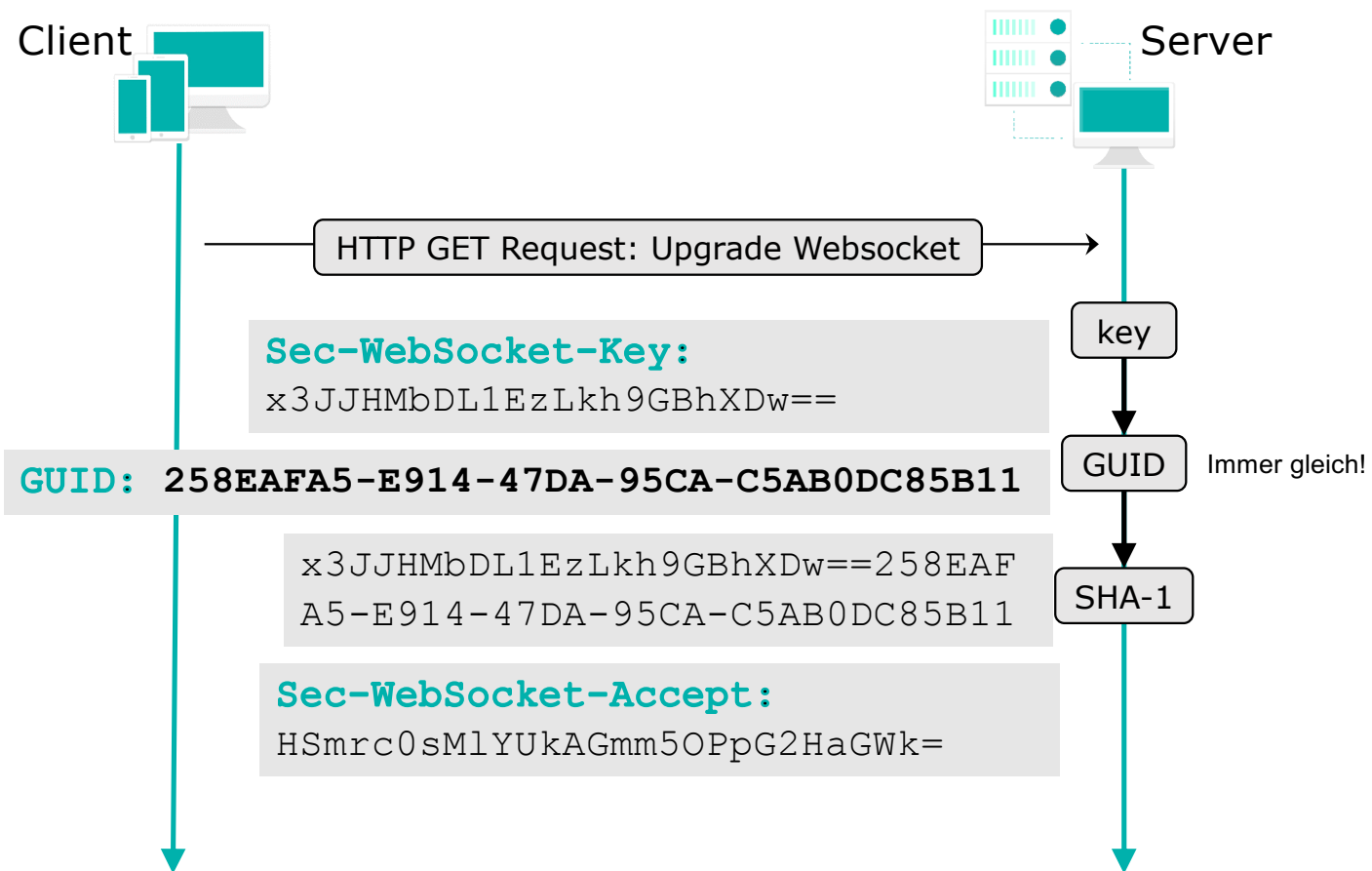
Node.js

Websockets Handshake



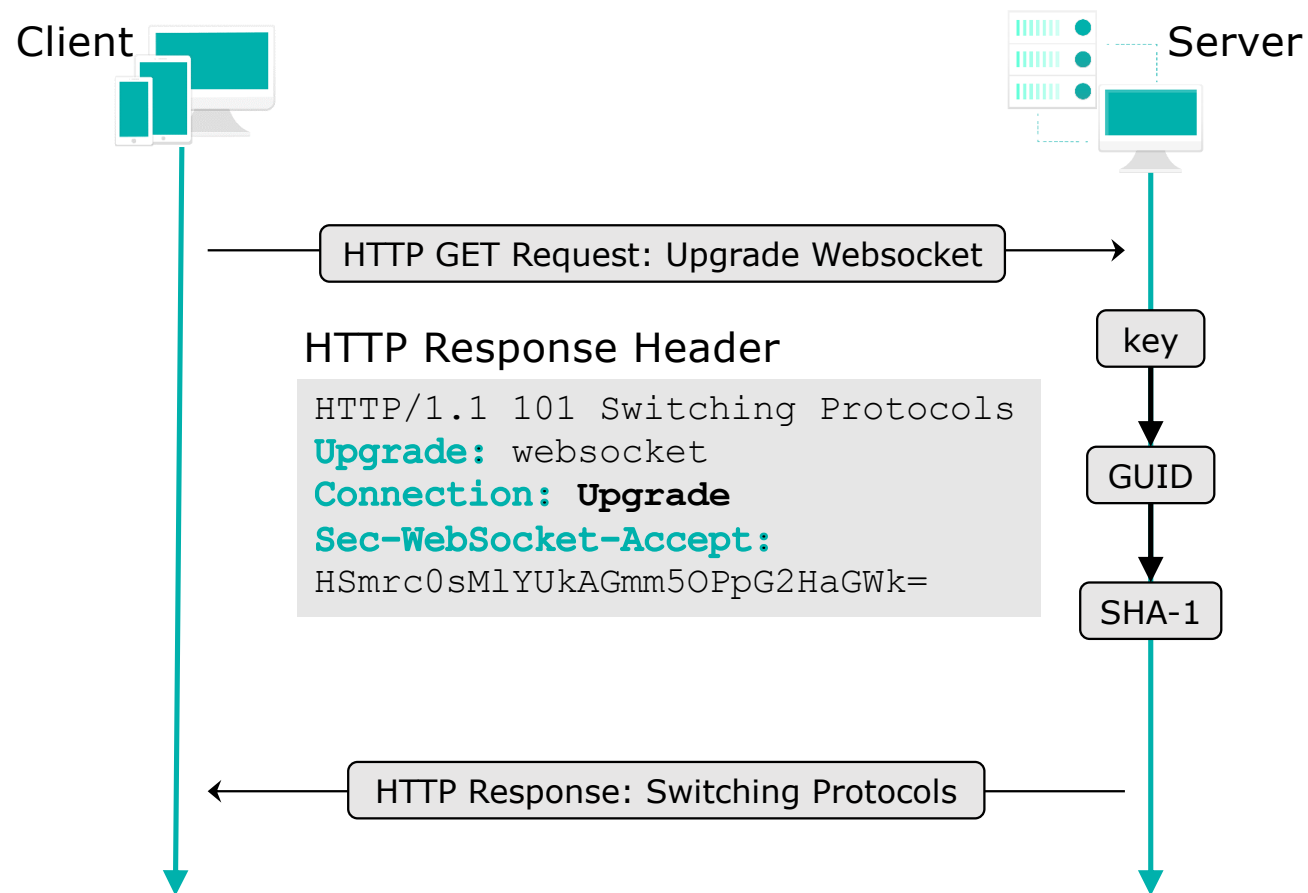
Node.js

Websockets Handshake



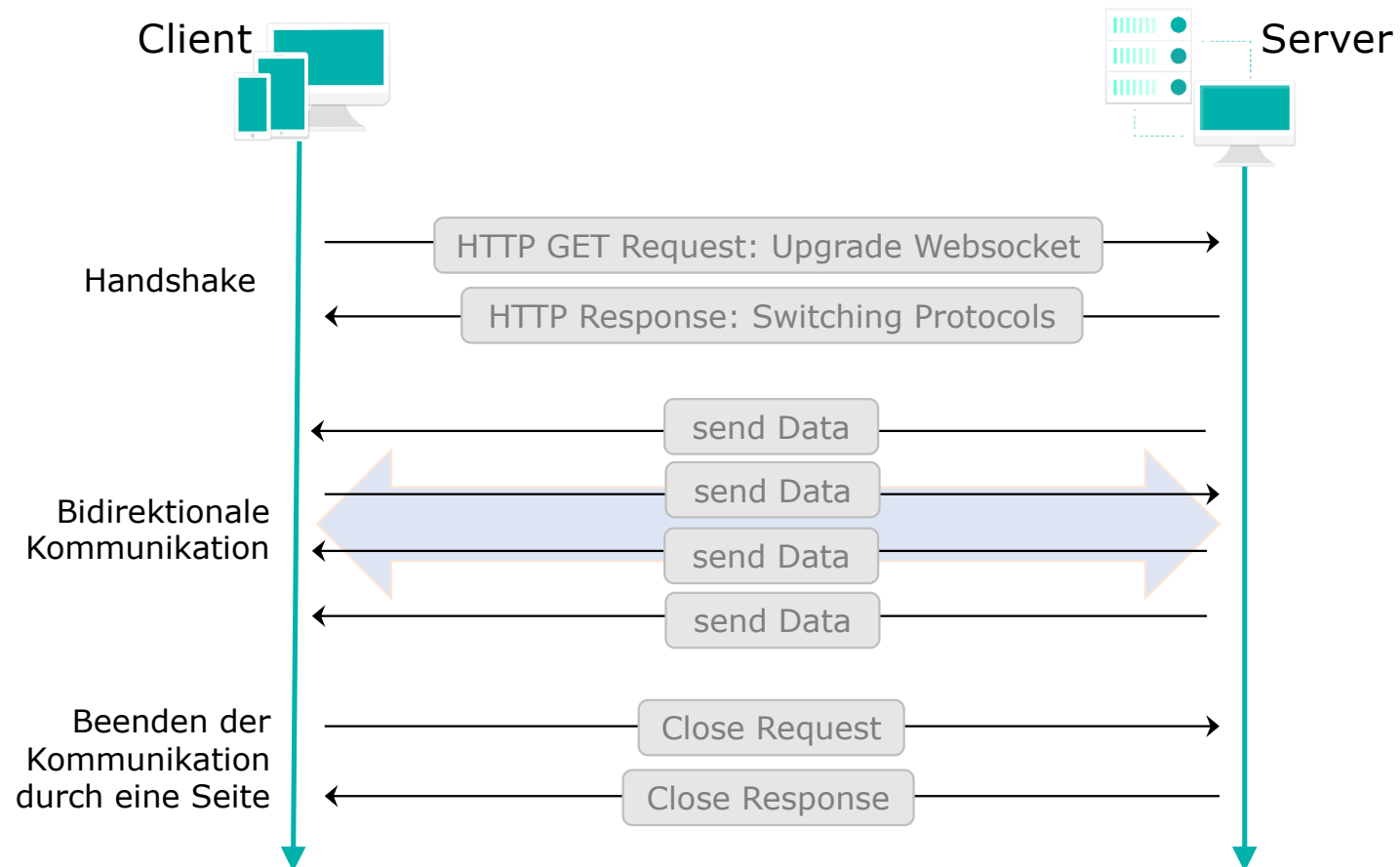
Node.js

Websockets Handshake



Node.js

Websockets Kommunikation



Node.js

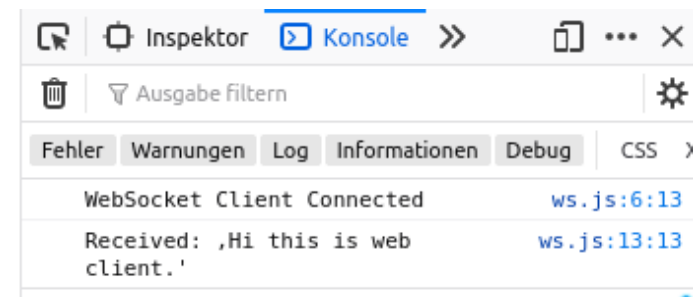
Hinweis zum Verbindungsupgrade

- Das Konzept eines Upgrades einer HTTP1.1-Verbindung auf etwas anderes ist HTTP2 und Websockets gemein
- Der damit verbundene Mehraufwand einer zusätzlichen Nachricht störte jedoch. Da heutzutage nahezu alle Verbindungen mit TLS durchgeführt werden wurde für HTTP der Trick eingeführt, dies durch **Application Layer Protocol Negotiation** zu lösen. Der eigentliche „Upgrade“ ist damit kein extra HTTP-Request mehr, sondern ein kurzer Teil des TLS-Handshakes.
- Websockets nutzen weiterhin den klassischen Wechsel durch Upgrade einer 1.1 Verbindung

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>WebSocket Playground</title>
  <script src="ws.js" defer></script>
</head>
<body>
  <p>Websocket Playground (Press F12)</p>
</body>
</html>
```

Websocket Playground (Press F12)



Node.js

Websockets - Client

ws.js (Client/Browser)

```
const socket = new WebSocket("ws://localhost:8080/");
//bei https muss wss verwendet werden

// Auf Verbindungsereigniss reagieren
socket.onopen = function () {
    console.log("WebSocket Client Connected");
    //senden von daten
    socket.send("Hi this is web client.");
}

// Auf Nachrichten reagieren
socket.onmessage = function (e) {
    console.log("Received: ," + e.data + "'");
};
```

Node.js

Websockets - Server

```
import express from 'express'
import { WebSocketServer } from 'ws'
import path from 'path'
import { fileURLToPath } from 'url';

// connectionHandler für Websocket in externer Datei
import connectionHandler from './websockets/connection.js'

const app = express();

// Einbinden der URL websockets __dirname funktioniert nicht
const __filename = fileURLToPath(import.meta.url)
const __dirname = path.dirname(__filename)
app.use(express.static(path.join(__dirname, 'websockets'))))

app.get('/', (req, res) => {
  res.sendFile('index.html', { root: __dirname })
})

app.get('/ws.js', (req, res) => {
  res.sendFile('ws.js', { root: __dirname })
})
```

Node.js

Websockets - Server

```
// wir haben keinen eigenen Server für Websockets und nutzen express
const wsServer = new WebSocketServer({ noServer: true });

// Aktivierung des connectionHandler
wsServer.on('connection', connectionHandler);

// Starte unseren Server - > Liefert http-Server-Objekt!
const server = app.listen(8080);

// Der folgende Code ist immer gleich
// Wenn wir ein Upgrade erhalten, dann verarbeiten wir 3 Parameter
server.on('upgrade', (request, socket, head) =>
  // Führe Upgrade aus: Hierzu wird der registrierte Event
  // 'connection' verwendet
  { wsServer.handleUpgrade(request, socket, head,
    socket => { wsServer.emit('connection', socket, request); }
  )
});
```

```
// './websockets/connection.js'
// Connection Handler (wird durch connection aufgerufen)

export function connectionHandler(socket, req) {

  console.log("Someone connected");

  socket.on('message', function (message) {
    console.log("Nachricht:" + message);
    //sendet Nachricht zurück
    socket.send(message);
  });
}
```